

cCKF: Calibrated Combinatorial Kalman Filter

Replacing Heuristic Cuts with Learned, Calibrated Decisions

Matthew Musson

Stanford University

August 28, 2026

Outline

Motivation

- The χ^2 Cut
- Why It Fails

Method

- Gate Function g_ψ
- Value Function V_ϕ
- Training

Results

- Inference Mechanics
- Bug Fixes
- Retraining
- Threshold Grid Search
- Tier 3 Value Targets
- Status

Further Work

What ACTS Does Today

- ▶ ACTS reconstructs tracks in three steps:
 1. Seeding
 2. The CKF
 3. Ambiguity Resolution
- ▶ The CKF cuts on the χ^2 value of a candidate hit which is a weighted version of the hits residual with respect to predicted measurement, weighted by the covariance of the Kalman filter

$$C_k^{k-1} = FC_{k-1}^{k-1}F^T + Q_k \quad (1)$$

$$S_k = H_k C_k^{k-1} H_k^T + R_k \quad (2)$$

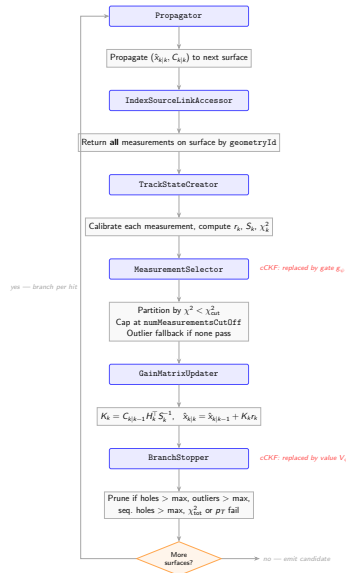
$$\hat{m}_k = H_k \vec{x} \quad (3)$$

$$r = h_k - \hat{m}_k \quad (4)$$

$$\chi^2 = r^T S^{-1} r \quad (5)$$

The CKF Algorithm

- ▶ The CKF applies a uniform cut to all hits, selecting those within a χ^2 threshold
- ▶ The CKF Branches on these hits up to numMeasurementsCutOff
- ▶ If there are no measurements within ChiSquaredCutOffMeasurement it checks for hits within chi2CutOffOutlier branching on those instead
- ▶ If no hits are within chi2CutOffOutlier then the CKF logs a hole
- ▶ Finally the CKF prunes branches which exceed a max count of holes, outliers, or sequential holes, as well as branches with a χ^2_{tot} or p_T which fail thresholds



Miscalibration

- ▶ This algorithm is insufficient in three main ways
 1. It is myopic with respect to detector occupancy - there is no account for local hit density (like in a jet core)
 2. The decision boundary exists only in χ^2 which is a compression of significantly more relevant signal (η , cluster features, etc.)
 3. The branch pruning is heuristic, not taking into account per measurement features
- ▶ A naive cut on χ^2 is blind to non-gaussian processes like hard scatters through detector material, requiring a learned discriminator

$$p(y = 1|x) \approx \frac{1}{\sqrt{2\pi \det(S_k)}} \exp \left[-\frac{1}{2} r_k^T S_k^{-1} r_k \right]$$

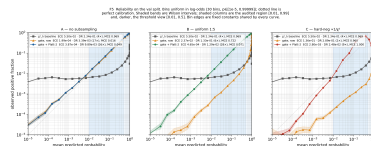


Figure: Miscalibration given implied Gaussian Probability

Window Failure Rate

placeholder
WindowFailureRateProcedure.png

Window Failure Rate

- We can see that hits fall out of the S_k weighted search window with accumulated material (data collected from optimized CKF config on ColliderML)

Window failure rate vs η — stratified by sensor type and seed purity



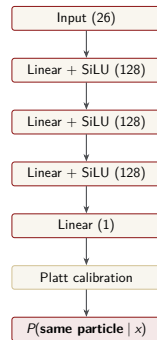
Learned Gate g_ψ

Answers: Is this hit from the same particle as the branch?

Feature vector (26-dim):

- ▶ Residual r_k (2D), Cholesky of S_k (3), χ_k^2
- ▶ Cluster: size, charge, second moments $\sigma_{uu}, \sigma_{uv}, \sigma_{vv}$
- ▶ Normalized cluster: $\kappa_u, \kappa_v, \tilde{Q}$
- ▶ Occupancy n_{window}
- ▶ Context: $\eta, q/p, \text{step } k, X/X_0$
- ▶ Sensor: $\text{pitch}_u, \text{pitch}_v, \text{thickness}, \text{is_pixel}$
- ▶ Branch history: $n_{\text{hits}}, n_{\text{holes}}, n_{\text{seq_holes}}$

Replaces: $\chi^2 < \chi_{\text{cut}}^2$



Loss: Unweighted BCE

$$\mathcal{L} = -\frac{1}{N} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

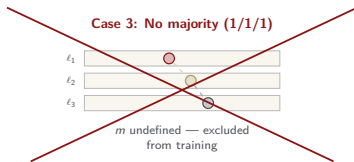
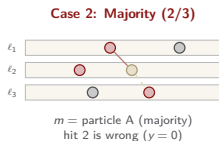
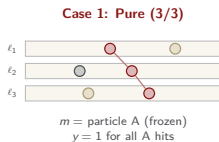
Incidence angle and expected charge



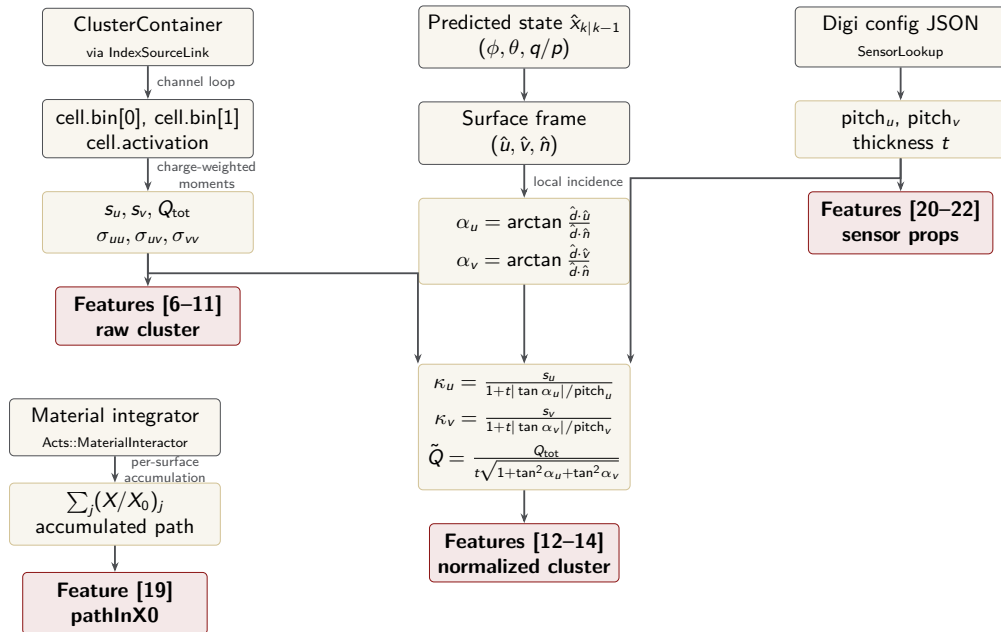
placeholder
incidenceAngle.png

Defining a Majority Particle

- ▶ In order for the gate function to be well defined we need to define a branch's majority particle
- ▶ In building seeding triplets there are three cases to consider
 - ▶ The triplet is pure, containing (3/3) hits from the same particle
 - ▶ The triplet has a majority, but one wrong hit, with purity (2/3)
 - ▶ The triplet is a mix of 3 different particles
- ▶ We exclude case 3 and train only on seeds with a majority particle — training a set of models on just pure triplets (which experience significantly lower window failure rates)



Feature Extraction Mechanics



Learned Value Function V_ϕ

Answers: Will this branch complete to a matched track?

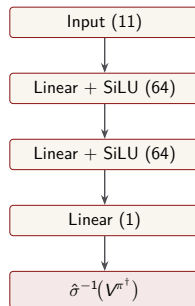
Feature vector (11-dim):

- ▶ Gate log-odds (accumulated)
- ▶ χ^2_{tot} , mean χ^2 per hit
- ▶ n_{hits} , n_{holes} , $n_{\text{seq_holes}}$
- ▶ η , q/p , p_T
- ▶ Step k , X/X_0 accumulated

Target: $V^{\pi^\dagger}$ (truth-greedy rollout)

$$V^{\pi^\dagger} = \min(\text{completeness}, \text{purity})$$

Replaces: hole/branch caps



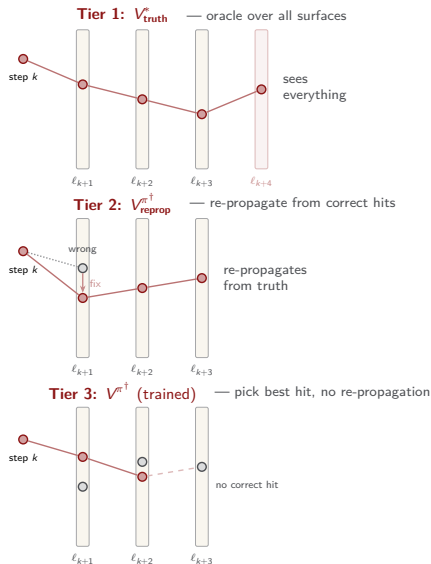
Loss: Soft-target BCE

$$\mathcal{L} = -\frac{1}{N} \sum_i [v_i \log \hat{v}_i + (1 - v_i) \log(1 - \hat{v}_i)]$$

where $v_i = V_i^{\pi^\dagger} \in [0, 1]$ is a soft label.

Three Tiers of the Value Function

- ▶ The value function predicts if a branch will complete to a DM-matched track
- ▶ Three tiers of oracle rollout, each a weaker upper bound:
 - ▶ Tier 1 (V^*): knows the true trajectory everywhere, even beyond CKF surfaces
 - ▶ Tier 2 ($V^{\pi^\dagger}_{\text{reprop}}$): picks correct hits on CKF surfaces, re-propagates from them (fixes corrupted Kalman state)
 - ▶ Tier 3 ($V^{\pi^\dagger}$): picks best available hit on each CKF surface, no re-propagation
- ▶ Tier 3 is the tightest bound the CKF can achieve without modifying the propagator



Training Six Models

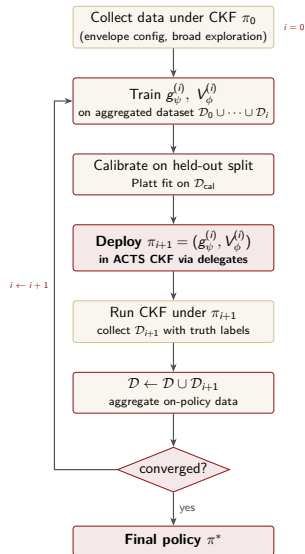
- ▶ We want to examine the performance of the gate function when trained on pure triplets or all majority defined seeds
- ▶ We also want to examine the difference between the tier 2 and tier 3 value functions
- ▶ Thus we are left to train six different models
- ▶ We run all of them on the test split to examine distribution shift when exposed to seeds which have purity of (2/3) and (3/3)

	Pure seeds (3/3)	Majority seeds (2/3)
Gate g_ψ	g_ψ^{pure} 26-dim, BCE $y = \mathbb{I}[\text{pid} = m]$	g_ψ^{maj} 26-dim, BCE $y = \mathbb{I}[\text{pid} = m]$
Value V_ϕ Tier 2	$V_\phi^{\text{pure}, T2}$ 11-dim, soft BCE $v = V_{\text{reprop}}^{\pi^\dagger}$	$V_\phi^{\text{maj}, T2}$ 11-dim, soft BCE $v = V_{\text{reprop}}^{\pi^\dagger}$
Value V_ϕ Tier 3	$V_\phi^{\text{pure}, T3}$ 11-dim, soft BCE $v = V_{\pi^\dagger}$	$V_\phi^{\text{maj}, T3}$ 11-dim, soft BCE $v = V_{\pi^\dagger}$

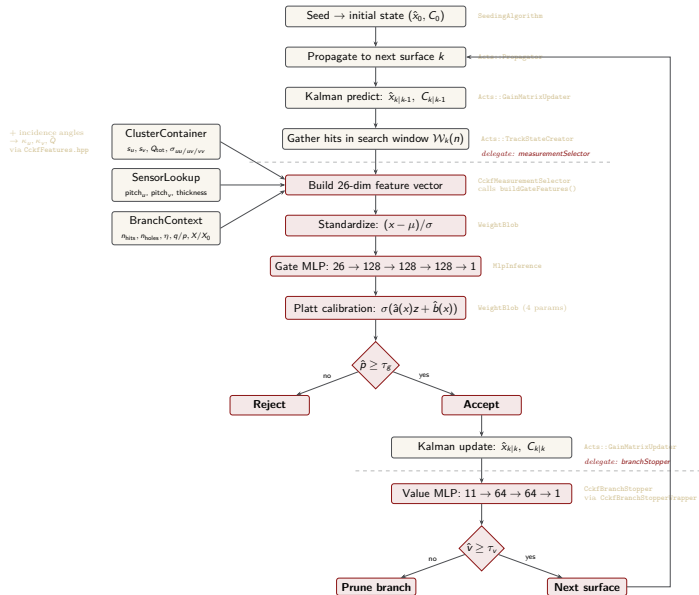
$2 \times 3 = 6$ models trained independently

Distribution Shift and DAgger

- ▶ The gate and value functions are initially trained under the CKF branches (ρ_{train})
- ▶ This means when they are rolled out, if they make wrong decisions, they'll enter into a different state distribution ρ_{deploy}
- ▶ The gate makes a wrong decision, the CKF accepts this decision, narrowing covariance, the next predicted measurement is wrong, χ^2 blows up...
- ▶ DAgger (Dataset augmentation) teaches the model to avoid these death spirals by retraining g_ψ, V_ϕ on its training distribution



cCKF Inference Pipeline



Two Bugs: NaN in S_{11} and Missing Strips

- ▶ **S_{11} NaN.** Strip sensors measure one coordinate, so their residual covariance is 1×1 . The feature builder read the full 2×2 block on every surface. On strips this produced NaN features and undefined behavior in Eigen expression proxies, which caused segfaults during inference.
- ▶ **Fix:** dimension-aware feature extraction, and every Eigen accessor result materialized with `.eval()` before use.
- ▶ **The strip bug.** Endcap geometry ids carry a ring index in their lowest byte. A join in the data pipeline zeroed that byte, so every endcap hit silently dropped out of the training set. The first gate was trained on barrel pixels only.
- ▶ **Fix:** normalize geometry ids in every loader. All 32 events were then re-expanded and every training cache rebuilt.

Distribution Shift at the Selector

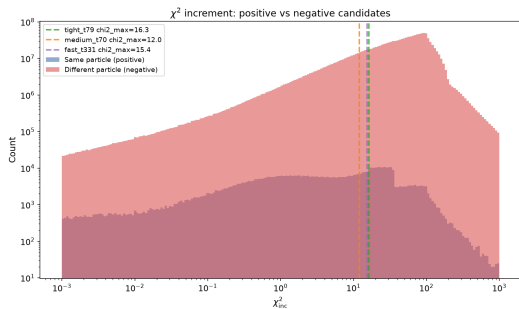


Figure: Training: χ^2 increments of gate candidates, with the tuned CKF cut values marked.

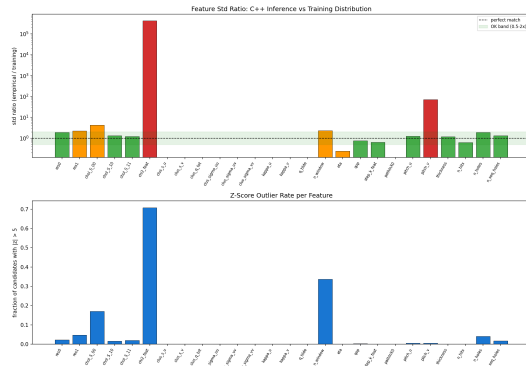


Figure: Inference: the selector receives every hit on the surface, so the χ^2 feature blows up relative to training. Fixed with a spatial pre-filter that reproduces the training window.

Retraining After the Fixes

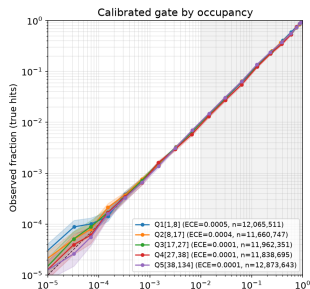
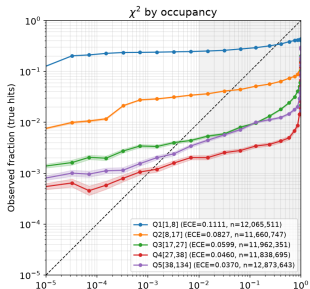
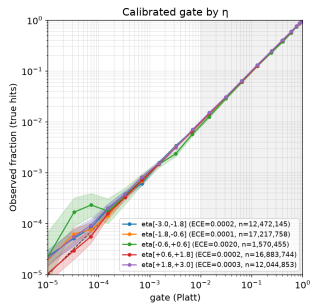
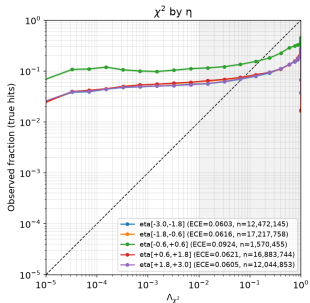
- ▶ All four models retrained on the rebuilt caches (full detector, endcaps included).

Model	AUC-ROC	AUC-PR	ECE	χ^2 ECE
Gate, majority seeds	0.989	0.852	0.0001	0.062
Gate, pure seeds	0.991	0.960	0.0010	0.059
Value T2, majority	0.822	0.535	0.0025	—
Value T2, pure	0.940	0.972	0.0082	—

- ▶ The majority gate is $129\times$ better calibrated than the χ^2 likelihood on the same calibration split.
- ▶ The value function sits 0.056 above the soft-target entropy floor, close to the irreducible limit.

Gate Calibration

G3: χ^2 -implied probability is miscalibrated and stratum-dependent; the calibrated gate is not



Occupancy Adaptation and Value Reliability

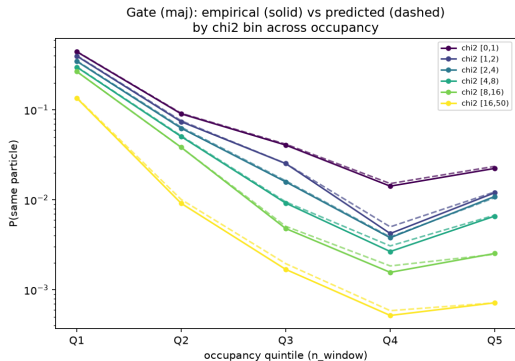


Figure: At fixed χ^2 , the true match probability falls $\sim 30\times$ from sparse to dense regions. The gate tracks it.

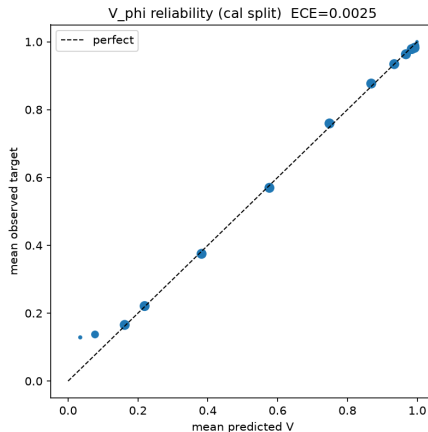
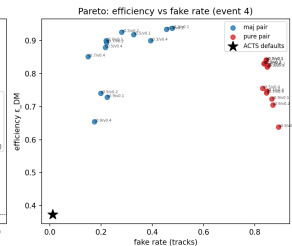
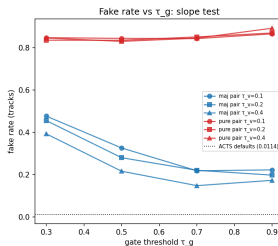


Figure: V_{ϕ} reliability on the calibration split (19M branch states), ECE 0.0025.

Grid Search: Pure vs Majority Seeds

- ▶ 12-point grid over (τ_g, τ_v) on a validation event, for each seed population.
- ▶ Majority pair: efficiency 0.94 at the loose end against the ACTS baseline of 0.37. Fake rate falls from 0.48 to 0.15 as the gate tightens, so the threshold behaves as a real likelihood-ratio test.
- ▶ Pure pair: fake rate is flat near 0.85 at every threshold.



Interpreting the High Fake Rates

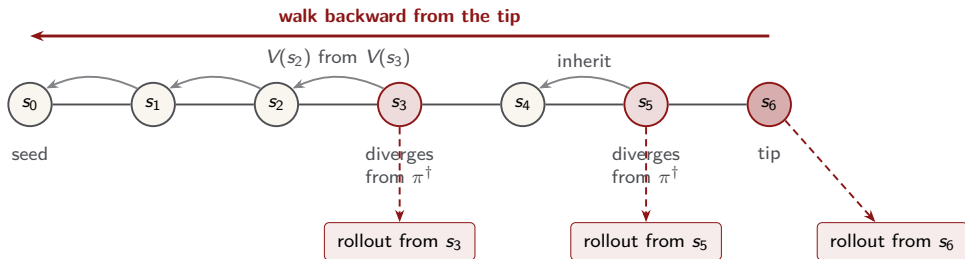
- ▶ The pure models never saw branches from majority seeds during training. On those branches their probabilities are meaningless, so the threshold stops discriminating and fakes pass at every setting. Calibration holds only on the training distribution.
- ▶ The majority pair responds correctly to its thresholds but its absolute fake rate still sits above the baseline. Two reasons:
 - ▶ The grid is coarse and has not reached the low-fake corner of threshold space.
 - ▶ Ambiguity resolution is unchanged from stock ACTS and does not yet use the calibrated scores, so fakes that reach it mostly survive.
- ▶ Efficiency more than doubles over the baseline at every grid point, so the trade is threshold placement, not model quality.

Tier 3: Value Targets by Re-propagation

- ▶ Tier 3 targets require rolling the CKF forward truth-greedily from every branch state. Naively that is one propagation per state, which is millions per event.
- ▶ **Backward collapse.** Walk each branch backward from its tip. Wherever the branch action matches the truth-greedy pick, the value follows from the child state by recurrence. Only divergence points and tips need a real rollout. This removes 77% of the propagations.
- ▶ **Worklist mechanic.** Each state that needs a rollout is written to a worklist row: seed id, step, surface id, bound parameters, covariance. A C++ executor seeds the ACTS propagator from each row and follows the branch's majority particle to termination, logging the hit sequence.
- ▶ Validated on one event: 419,704 of 419,704 rollouts completed with zero surface lookup failures. Generation for all 32 events is queued; target composition and training follow.

Backward Stitching

- One branch, walked tip to seed. Rollouts run only where they are needed; everything else is stitched by recurrence.



- action matches $\pi^\dagger$: value by recurrence
- divergence or tip: value from a real rollout

Current Runs

- ▶ **EHVI Pareto sweeps.** Adaptive threshold search with expected hypervolume improvement, warm-started from the grids. Five rounds of eight points per seed population, running on Perlmutter.
- ▶ **Window scan.** The strongest grid points rerun at search window sizes $n \in \{7, 5, 3\}$ to separate the speedup from shrinking the window from the accuracy cost of leaving its training value.
- ▶ **Tier 3 generation.** Worklist and rollout jobs for all 32 events.
- ▶ **Dagger data.** Every sweep run logs its track states, so on-policy training data accumulates for free.

Next Week

- ▶ Finish EHVI Pareto fronts for all four pairs: $\text{gate} \times \{\text{majority, pure}\}$ with value tier 2, then tier 3.
- ▶ Train and benchmark both tier 3 value functions against tier 2.
- ▶ Make the DAgger call: measure calibration drift on the logged on-policy states. Iterate only if the drift is material.
- ▶ Use the calibrated scores in ambiguity resolution and remeasure the fake rate.